PROJECT_STATE.md

OVERWATCH — Master Project State & Execution Roadmap

Project: Overwatch
Document Version: 1.0
Status: Phase 0 — Discovery & Architecture
Development Model: Local → `dev` → Testing/UAT → `main` → Production
Primary Goal: Build a secure, reliable, low-cost operational intelligence platform that works alongside Housecall Pro and eventually becomes the company's primary operational interface.

---

1. DOCUMENT PURPOSE

This document is the **source of truth for the Overwatch project**.

It exists to prevent:

- Lost ideas
- Lost requirements
- Rebuilding completed work
- Moving forward before a phase is actually finished
- Security being added too late
- Important decisions being forgotten
- Uncontrolled feature creep
- Deploying something that only works on a developer machine
- Building large portions of the system without proving they work
- Creating code that the project owner cannot understand later

This document will be updated throughout development.

Golden Rule

> **We do not move to the next phase because the current phase is "mostly done."**
>
> We move forward only when the current phase meets its Definition of Done and has been accepted.

---

2. HOW THE ASSISTANT SHOULD WORK

The assistant is acting as:

- Senior Full-Stack Developer
- Software Architect
- Database Architect
- Integration Engineer
- Security Advisor
- UI/UX Architect
- Testing Engineer
- DevOps Advisor
- AI Systems Architect
- Technical Teacher

The assistant should not simply provide code.

For important decisions, explain:

1. What we are doing
2. Why we are doing it
3. What problem it solves
4. What alternatives exist when relevant
5. What risks exist
6. What we should test
7. How it affects future phases

Development Teaching Requirement

The project owner wants to understand the system, not simply receive a finished application.

Therefore:

- Code should be readable.
- Important code should contain useful comments.
- Architecture should be explained.
- Complex functions should explain their purpose.
- Security-sensitive code should explain important assumptions.
- Integration code should explain what system it communicates with.
- Database relationships should be understandable.
- AI logic should explain what it is allowed and not allowed to do.

The goal is:

> **Build the application and teach the developer how the application works at the same time.**

3. CODE COMMENTING & LEARNING STANDARD

This is a permanent project requirement.

Comments Are Required When They Add Understanding

Comments should explain:

- Why something is being done
- Why a particular approach was chosen
- Important business rules
- Security considerations
- Integration assumptions
- Non-obvious calculations
- State transitions
- AI decision boundaries
- Offline/synchronization behavior
- Database relationships
- Important edge cases

Example

Good:

```
// HCP remains the system of record for this job.
// Overwatch stores a local copy so the technician can continue
// working if the device temporarily loses connectivity.
```

Less useful:

```
// Set job equal to job.
const job = job;
```

We do not want comments that merely repeat what the code already says.

Complex Functions

Important functions should have a short explanation of:

- What the function does
- What it expects
- What it returns

- Important side effects
- Important failure conditions

Security-Sensitive Code

Security-sensitive areas should receive especially clear comments explaining:

- What is being protected
- What authorization is being checked
- Why the check exists
- What could happen if the check were removed

Business Logic

Business rules should be easy to locate and understand.

Examples:

- Job state transitions
- Pricing permissions
- Discount limits
- Part request rules
- Cancellation reasons
- Technician skill requirements
- Scheduling constraints
- AI automation permissions

TODO Comments

TODO comments are allowed, but they must be meaningful.

Example:

```
// TODO: Replace temporary local pricing lookup with the production
// pricing service once the HCP pricing integration is available.
```

Do not use vague TODOs such as:

```
// TODO: fix this
```

Code Should Teach

When reasonable, code should favor clarity over cleverness.

Avoid unnecessary:

- Abstractions
- One-line magic
- Extremely compressed code
- Unexplained patterns
- Over-engineering
- Dependencies that provide little value

The project should remain understandable to the person maintaining it.

---

4. PROJECT PHILOSOPHY

Overwatch will be built around several principles.

4.1 Technician First

The technician experience is the first major product.

The technician should not need to understand the underlying architecture.

The application should guide them through the work.

4.2 Enter Information Once

If a technician enters information into Overwatch, they should not have to enter the same information again elsewhere.

Overwatch should structure the information and synchronize it to the appropriate systems.

4.3 HCP Remains the System of Record Initially

Housecall Pro remains the authoritative operational system initially.

Overwatch adds:

- Intelligence
- Workflow
- Better interfaces
- Structured information
- Automation
- Analytics
- Cross-system coordination

Overwatch does not attempt to replace HCP on day one.

4.4 Deterministic Software for Facts

Traditional software should handle:

- Exact calculations
- Permissions
- State transitions
- Pricing rules
- Inventory quantities
- Required fields
- Scheduling constraints
- Security
- Authentication
- Authorization

AI should not be responsible for exact facts when deterministic software can handle them.

4.5 AI for Understanding

AI is appropriate for:

- Understanding natural language
- Extracting information
- Classifying notes
- Summarizing history
- Identifying patterns
- Recommendations
- Predictive analysis
- Interpreting technician voice notes

AI must not become an uncontrolled source of truth.

---

5. COST PHILOSOPHY

Overwatch should operate as cheaply as realistically possible.

Priority:

1. Free
2. Open source
3. Self-hosted where practical
4. Existing HCP capabilities
5. Free tiers

6. Low-cost services
7. Paid services only when they provide significant value

Every new external service should be evaluated for:

- Cost
- Free tier
- Long-term cost
- API availability
- Vendor lock-in
- Reliability
- Security
- Data ownership
- Ease of replacement

Third-Party Dependency Rule

Before adding a paid dependency, document:

- What it does
- Why we need it
- Free alternatives considered
- Estimated cost
- What happens if the service disappears
- Whether the architecture can replace it later

---

6. DEVELOPMENT ENVIRONMENT

Primary development tools:

- VS Code
- GitHub
- Local development environment

The project must remain easy to run.

Target:

```
npm install
npm run dev
```

No required `setup.sh`.

Additional commands may exist for testing, building, migrations, etc., but basic development should remain straightforward.

---

7. GIT WORKFLOW

Exactly two primary branches:

`dev`

Development and testing.

All new work goes here first.

`main`

Production.

`main` represents code approved for production.

Workflow

```
LOCAL DEVELOPMENT
       ↓
      dev
       ↓
FULL TESTING
       ↓
USER ACCEPTANCE
       ↓
MERGE TO main
       ↓
PRODUCTION
       ↓
PRODUCTION VERIFICATION
```

Never experiment directly on `main`.

---

8. DEVELOPMENT STRATEGY

We will use **vertical slices** whenever practical.

Instead of building:

```
Entire database
↓
Entire API
↓
Entire frontend
↓
Entire security system
↓
Entire testing system
```

we will build complete pieces of functionality.

Example:

```
Technician Login
    ↓
Today's Jobs
    ↓
Open Job
    ↓
Job Details
    ↓
Change Job State
    ↓
Save
    ↓
Security
    ↓
Testing
    ↓
Deploy
    ↓
Verify
```

This produces usable software earlier and exposes architectural problems earlier.

---

9. UNIVERSAL PHASE PROCESS

Every phase follows the same lifecycle.

STEP 1 — PLAN

Define:

- Objective
- Requirements
- Dependencies
- Risks
- Security requirements
- Data requirements
- UI requirements
- Testing requirements
- Exit criteria

STEP 2 — DESIGN

Determine:

- Architecture
- Database changes
- API changes
- UI flow
- Permissions
- Integration behavior
- Error handling
- Offline behavior where applicable

STEP 3 — BUILD

Implement the smallest complete version that satisfies the phase.

STEP 4 — SECURITY

Security is implemented during the phase.

Not afterward.

Review:

- Authentication
- Authorization
- Data exposure
- Input validation
- Secrets
- Logging
- Permissions

- API access
- File access
- Privacy

STEP 5 — TEST

Test:

- Happy path
- Failure path
- Edge cases
- Permission boundaries
- Mobile behavior
- Offline behavior where applicable
- Integration behavior
- Recovery behavior

STEP 6 — USER ACCEPTANCE

The project owner tests the actual workflow.

A feature is not complete simply because the developer says it works.

STEP 7 — DOCUMENT

Update:

- PROJECT_STATE.md
- README where necessary
- Architecture documentation where necessary
- API documentation where necessary
- Important code comments

STEP 8 — DEPLOY

Deploy the completed phase to the appropriate environment.

STEP 9 — VERIFY

Verify the deployed version.

STEP 10 — MARK COMPLETE

Only after all exit criteria are satisfied.

---

10. DEFINITION OF DONE

A phase is NOT complete because:

- The page loads
- The code compiles
- The API works once
- It works on the developer's computer
- The UI looks good
- The developer believes it is finished

A phase is complete only when all applicable requirements below are satisfied.

Functionality

- [ ] Requirements implemented
- [ ] Normal workflow works
- [ ] Error workflow works
- [ ] Edge cases considered
- [ ] Data is saved correctly
- [ ] Data is retrieved correctly

Security

- [ ] Authentication implemented where required
- [ ] Authorization implemented where required
- [ ] User permissions verified
- [ ] Input validated
- [ ] Sensitive data protected
- [ ] Secrets not exposed
- [ ] API endpoints protected
- [ ] Security failure cases tested

Reliability

- [ ] Errors handled
- [ ] Failed requests handled
- [ ] Retry behavior considered
- [ ] Duplicate actions considered
- [ ] Data integrity protected
- [ ] Recovery behavior tested

UX

- [ ] Workflow is understandable
- [ ] Mobile behavior tested where applicable

- [ ] Loading states exist
- [ ] Error states exist
- [ ] Empty states exist
- [ ] User feedback exists
- [ ] User does not have to unnecessarily repeat information

Testing

- [ ] Developer testing completed
- [ ] Automated tests added where appropriate
- [ ] Integration tests added where appropriate
- [ ] Manual testing completed
- [ ] Failure cases tested
- [ ] Permission testing completed

Documentation

- [ ] PROJECT_STATE.md updated
- [ ] Important code commented
- [ ] README updated if needed
- [ ] Important architectural decisions recorded

Deployment

- [ ] Builds successfully
- [ ] Deploys successfully
- [ ] Production/staging behavior verified
- [ ] Rollback path understood

Acceptance

- [ ] User has tested it
- [ ] User accepts the phase
- [ ] No known blocker remains

---

11. SECURITY-BY-DESIGN

Security is a cross-cutting requirement.

It is NOT Phase 25 only.

Security begins with the first authentication screen and continues through every feature.

Every phase asks:

Authentication

Who is this user?

Authorization

What is this user allowed to do?

Data Access

What data is this user allowed to see?

Data Modification

What data is this user allowed to change?

Auditability

Can we determine who changed something?

Privacy

Should this information be visible to this person?

Integration Security

Can external systems safely communicate with Overwatch?

Secrets

Are API keys, tokens, credentials, and private configuration protected?

---

12. IDENTITY & ROLE MODEL

Initial role concepts:

- Technician
- Dispatcher
- Sales
- Manager
- HR/People
- Warehouse/Parts
- Administrator

Permissions should be capability-based where practical rather than relying solely on job titles.

Example:

```
CanViewJob
CanUpdateJob
CanRequestParts
CanApprovePricing
CanViewEmployeeLocation
CanViewCustomerHistory
CanManageUsers
CanManageSystemSettings
```

This makes future permission changes easier.

---

13. MASTER ROADMAP

PHASE 0 — DISCOVERY & ARCHITECTURE

Objective

Understand the problem completely enough to build the correct foundation.

Work

- Finalize Overwatch vision
- Define system boundaries
- Research HCP APIs
- Research HCP webhooks
- Research HCP messaging
- Research HCP GPS capabilities
- Research HCP customer portal capabilities
- Research Mobile N Client / DSI functionality
- Evaluate communication options
- Evaluate GPS options
- Evaluate PWA vs native
- Evaluate database options
- Evaluate hosting options
- Define authentication architecture
- Define authorization model
- Define data ownership
- Define event model
- Define offline architecture
- Define initial AI boundaries

Exit Criteria

- [ ] Architecture documented
- [ ] Major dependencies understood
- [ ] Technology choices documented
- [ ] Major risks documented
- [ ] HCP integration strategy documented
- [ ] Security architecture direction documented
- [ ] First vertical slice defined

PHASE 1 — PROJECT FOUNDATION

Objective

Create the production-quality application foundation.

Build

- Repository
- React/frontend foundation
- Backend/API foundation
- Database foundation
- Environment configuration
- Logging
- Error handling
- Basic testing framework
- Development documentation
- Deployment foundation

Security

- Secrets management
- Secure environment configuration
- Basic API protection
- Security headers where applicable
- Dependency/security scanning where practical

Exit Criteria

A clean new developer should be able to:

```
npm install
npm run dev
```

and understand the project structure.

PHASE 2 — IDENTITY & PERMISSIONS

Objective

Establish secure user identity.

Build

- Login
- Logout
- Session management
- User records
- Roles
- Permissions
- Protected routes
- Protected APIs
- Account state

Security

- Password/security strategy
- Session protection
- Authorization checks
- Unauthorized access handling
- Audit events

Exit Criteria

A user can securely log in and only access what they are authorized to access.

PHASE 3 — HCP INTEGRATION FOUNDATION

Objective

Create the communication layer between Overwatch and HCP.

Build

- HCP authentication/integration
- Customer synchronization
- Job synchronization
- Technician synchronization
- Webhook receiver
- Event processing

- Error handling
- Sync status
- Retry handling

Critical Principle

HCP remains the source of truth initially.

Exit Criteria

Overwatch can reliably receive and synchronize required HCP information without creating duplicate records.

PHASE 4 — TECHNICIAN MY DAY

Objective

Give the technician a useful daily operational interface.

Build

- Today's jobs
- Job list
- Job details
- Customer
- Address
- Job description
- Job history
- Equipment information where available
- Schedule
- Basic navigation

Exit Criteria

A technician can open Overwatch and understand their day without needing to hunt through multiple screens.

PHASE 5 — JOB STATE MACHINE

States

```
ASSIGNED
↓
```

```
ACKNOWLEDGED
↓
EN_ROUTE
↓
ARRIVED
↓
DIAGNOSING
↓
WORKING
↓
WAITING
↓
COMPLETED
```

Alternative outcomes:

```
CANCELED
DECLINED
UNABLE_TO_COMPLETE
RESCHEDULE_REQUIRED
```

Requirements

Every state transition must:

- Be authorized
- Be recorded
- Have appropriate timestamps
- Handle invalid transitions
- Synchronize appropriately

Critical Rule

A job must never simply disappear into "not completed."

The system must know why.

PHASE 6 — GPS & ETA

Build

- Technician location
- Current job
- Next job

- ETA
- Travel status
- Arrival detection
- Last known location
- Schedule risk

Privacy

Customer-facing location must be appropriately limited.

Exact employee location must be permission-controlled.

Exit Criteria

Location information provides useful operational value without creating unnecessary privacy exposure.

PHASE 7 — JOB DOCUMENTATION

Build

- Voice notes
- Text notes
- Photos
- Files
- Job observations
- Customer notes
- Equipment details
- Completion documentation
- Signatures where applicable

AI

AI may interpret voice/text into structured information.

Example:

> "Door is worse than expected and the spring is broken."

Could become:

```
Problem:
Broken spring

Scope:
Greater than expected
```

```
Potential Parts:
Spring

Potential Follow-up:
Additional work estimate
```

AI extraction must remain reviewable.

PHASE 8 — PARTS SYSTEM

Build

- Part search
- Part number
- Description
- Category
- Availability
- Quantity
- Customer price
- Cost where permitted
- Parts used
- Parts needed
- Parts requested
- Parts ordered
- Backordered
- Unknown part identification
- Part photos

Exit Criteria

A technician can identify and record the parts involved in a job without leaving the job workflow.

PHASE 9 — PRICING

Build

- Part pricing
- Labor
- Estimates
- Customer pricing
- Discount permissions
- Pricing overrides

- Special pricing
- Approval requests

Security

Pricing permissions must be enforced server-side.

The UI cannot be the only protection.

PHASE 10 — PART REQUEST / OFFICE ASSISTANCE

Build

Technician can request:

- Part
- Pricing approval
- Second technician
- Office assistance
- Part identification
- Customer approval
- Additional equipment
- Scheduling assistance

The office receives the request with context automatically.

PHASE 11 — JOB OUTCOME INTELLIGENCE

Objective

Understand exactly why jobs do not get completed.

Structured Reasons

Examples:

- Customer canceled
- Customer unavailable
- Customer declined
- Price
- Went with competitor
- Fixed themselves
- Scheduling issue

- Duplicate job
- Technician skill limitation
- Wrong job description
- Wrong part
- Missing part
- Second technician required
- Specialty equipment required
- Unsafe
- Office approval required
- Time ran out
- Technician unable to complete
- Other

"Other" requires an explanation.

AI

AI can interpret natural language and suggest structured reasons.

The final structured reason must remain reviewable.

PHASE 12 — COMMUNICATION

Build

Contextual job communication:

- Text
- Push notifications
- Voice
- Push-to-talk where infrastructure supports it
- Photos
- Files
- Video/office call where practical
- Emergency/help request

The office should automatically receive job context.

Research Before Locking Infrastructure

Evaluate:

- HCP messaging
- Twilio
- WebRTC
- Zello

- Microsoft Teams
- Open-source alternatives
- Custom communication infrastructure

PHASE 13 — OFFLINE-FIRST FIELD OPERATION

Objective

A technician should remain productive when connectivity is poor.

Offline Data

Cache:

- Day's jobs
- Job details
- Customer information
- Relevant history
- Parts information where practical

Offline Actions

Queue:

- Notes
- Photos
- State changes
- Parts
- Check-in/out
- Documentation

Sync

```
DEVICE
 ↓
LOCAL DATA
 ↓
OFFLINE QUEUE
 ↓
CONNECTION RETURNS
 ↓
SYNC ENGINE
 ↓
OVERWATCH
```

```
  ↓
HCP
```

Must handle:

- Duplicate prevention
- Retry
- Conflicts
- Partial failures
- Recovery

PHASE 14 — CUSTOMER EXPERIENCE

Build

Appointment-specific customer experience:

- Technician status
- ETA
- Approximate map location
- Technician name/photo
- Appointment updates
- Communication
- Relevant documentation

Privacy remains a primary requirement.

PHASE 15 — OVERWATCH EVENT BUS

Objective

Create the operational nervous system.

Examples:

```
JOB_CREATED
JOB_UPDATED
JOB_SCHEDULED
JOB_RESCHEDULED
JOB_STARTED
JOB_COMPLETED
JOB_CANCELED
```

```
TECH_EN_ROUTE
TECH_ARRIVED
TECH_DELAYED
PART_REQUESTED
PART_USED
PART_UNAVAILABLE
TECH_HELP_REQUESTED
SECOND_TECH_REQUESTED
CUSTOMER_CANCELED
EMAIL_RECEIVED
DOCUMENT_UPLOADED
PTO_REQUESTED
PTO_APPROVED
ORDER_CREATED
ORDER_RECEIVED
```

Events allow Overwatch to determine:

> Does this matter?

Then:

```
NOTIFY
RECOMMEND
PREPARE
ACT
```

PHASE 16 — AI INTAKE

Objective

Turn messy information into structured operational information.

Sources:

- Emails
- Job descriptions
- Technician notes
- Voice
- Customer requests
- Internal messages
- Documents

AI extracts:

- Customer
- Problem
- Equipment
- Urgency
- Required skills
- Parts
- Scheduling requirements
- Risks
- Follow-up requirements

PHASE 17 — AI WATCHDOG

Objective

Continuously monitor operations.

Potential examples:

- Job at risk
- Technician running late
- Part delay
- Missing documentation
- Customer waiting
- Repeated failed visits
- Skill mismatch
- Schedule conflict
- Unusual cancellation
- Pricing problem
- Communication failure

AI should recommend actions, not silently take dangerous actions.

PHASE 18 — AI ASSISTANT

Employees can ask:

> Why is this job at risk?
>
> What has this customer had done before?
>
> Which technician is best suited for this job?

Why are we having so many return trips?

What parts are causing delays?

What happened with this customer?

Answers must be grounded in Overwatch data.

PHASE 19 — SCHEDULING INTELLIGENCE

Scheduling should consider:

- Skills
- Certifications
- Availability
- PTO
- Location
- Travel time
- Job duration
- Historical performance
- Parts
- Equipment
- Customer requirements
- Priority
- Existing schedule

The system should first recommend.

Automation comes later.

PHASE 20 — LEARNING FROM OPERATIONS

Overwatch begins comparing:

```
PREDICTED
vs
ACTUAL
```

Examples:

- Estimated job duration vs actual
- Estimated travel vs actual

- Predicted parts vs actual
- Recommended technician vs outcome
- Predicted cancellation risk vs actual
- Predicted skill match vs result

This creates the feedback loop.

PHASE 21 — OFFICE OVERWATCH

Create operational dashboards for:

Dispatch

- Jobs
- At-risk jobs
- Technician locations
- Requests
- Schedule
- Recommendations

Sales

- Requests
- Quotes
- Follow-ups
- Lost opportunities

Management

- Company health
- Trends
- Risks
- Recommendations
- Performance

PHASE 22 — EMPLOYEE / COMPANY PORTAL

Long-term employee experience inspired by systems such as UKG.

Potential areas:

- Employee profile
- Schedule
- PTO

- Time
- Training
- Certifications
- Documents
- Company announcements
- Tasks
- Employee self-service

PHASE 23 — AUTOMATION

Introduce automation gradually.

Level 0 — Observe

System watches.

Level 1 — Inform

System alerts.

Level 2 — Recommend

System suggests.

Level 3 — Prepare

System prepares an action.

Level 4 — Execute Approved Actions

System automatically executes explicitly approved low-risk actions.

Level 5 — Autonomous

Only for carefully authorized categories.

PHASE 24 — AUDITABILITY

Every important automated or AI-assisted action should record:

- What happened
- Who initiated it
- Timestamp
- Source data
- AI recommendation
- Reason
- Confidence
- Approval
- Action taken
- Result
- HCP synchronization result

PHASE 25 — SECURITY HARDENING

Security has already existed throughout development.

This phase is a formal security review/hardening pass.

Review:

- Authentication
- Authorization
- API security
- Database security
- Secrets
- File access
- Logging
- Audit trails
- Privacy
- Employee location
- Customer data
- AI data access
- Third-party integrations
- Dependency vulnerabilities
- Backup/recovery
- Incident response

PHASE 26 — COMPLETE TESTING STRATEGY

Testing should exist throughout development.

Final testing expands into:

- Unit testing
- Integration testing
- API testing
- Database testing
- Authentication testing
- Authorization testing
- Security testing
- Mobile testing
- Offline testing
- Synchronization testing
- HCP integration testing
- AI evaluation
- Performance testing
- Failure/recovery testing
- Regression testing

PHASE 27 — USER ACCEPTANCE TESTING

The actual user performs realistic workflows.

Examples:

Technician

```
Login
↓
My Day
↓
Open Job
↓
Navigate
↓
Arrive
↓
Diagnose
↓
Add Parts
```

```
↓
Pricing
↓
Request Help
↓
Complete
↓
Documentation
↓
Submit
```

Dispatcher

```
Login
↓
Operations
↓
See At-Risk Job
↓
Review Technician
↓
Review Recommendation
↓
Take Action
```

UAT failures return the phase to development.

PHASE 28 — PRODUCTION READINESS

Verify:

- Environment configuration
- Secrets
- Database
- Backups
- Monitoring
- Logging
- Error tracking
- Performance
- Security
- Deployment
- Rollback
- Documentation
- Support procedures

PHASE 29 — DEPLOYMENT

Production deployment occurs only after approval.

Deployment must be:

- Repeatable
- Documented
- Recoverable
- Verifiable

PHASE 30 — MONITORING

Production Overwatch must be observable.

Monitor:

- Application errors
- API failures
- HCP synchronization
- Authentication failures
- Database health
- Queue failures
- Offline sync failures
- AI failures
- Performance
- Automation actions

---

14. FIRST VERTICAL SLICE

Before building the entire platform, prove the architecture with a complete small workflow.

First Slice

```
Technician Login
        ↓
Today's Jobs
        ↓
Open Job
```

```
        ↓
View Customer / Address / Description
        ↓
Change Job State
        ↓
Save State
        ↓
Authorization Check
        ↓
Database
        ↓
Event
        ↓
HCP Synchronization
        ↓
Audit Record
```

This slice must be:

- Secure
- Tested
- Documented
- Deployable
- Understandable
- Usable on the intended technician device

This is the first major architectural proof.

---

15. DATA MODEL — HIGH LEVEL

Expected major entities include:

```
User
Employee
Role
Permission

Customer
Address
Property

Job
Appointment
JobState
```

```
JobEvent
JobOutcome

Technician
TechnicianSkill
Certification
Availability

Part
Inventory
PartUsage
PartRequest
PartOrder

Price
Estimate
PricingApproval

Communication
Message
Attachment

Location
ETA

Document
Photo
VoiceNote

Notification

Event

AIInteraction
AIRecommendation
AIAction

AuditLog
IntegrationSync
```

This list is intentionally not the final database schema.

The schema will be designed as each phase requires it.

---

16. DATA OWNERSHIP

For every important data type, Overwatch must know:

- Where it originated
- Who owns it
- Whether Overwatch can modify it
- Where it synchronizes
- What happens when systems disagree

Example:

```
HCP Job
    ↓
Overwatch copy
    ↓
Technician updates Overwatch
    ↓
Overwatch validates
    ↓
HCP synchronization
```

---

17. AI RULES

AI must never silently:

- Invent customer information
- Invent pricing
- Invent inventory
- Override permissions
- Override hard scheduling constraints
- Change important records without authorization
- Hide uncertainty
- Make high-risk decisions without an approved workflow

AI should communicate uncertainty.

Example:

```
Recommended technician:
John
```

```
Confidence:
87%

Reason:
- Certified for equipment type
- Completed 18 similar jobs
- Average duration 12% below company average
- Currently 14 minutes away
```

---

18. CHANGE CONTROL

Changes WILL happen.

That is expected.

A new idea does not mean the roadmap is broken.

When a new requirement appears:

1. Record it.
2. Determine which phase it belongs to.
3. Determine whether it changes an existing decision.
4. Determine dependencies.
5. Determine security implications.
6. Determine cost implications.
7. Determine whether completed work is affected.
8. Update the roadmap.
9. Continue without unnecessary rework.

Do Not

- Randomly add features
- Rewrite completed systems without reason
- Change architecture because a new library looks interesting
- Add dependencies without evaluating them
- Skip phase gates

---

19. DECISION LOG

Important architectural decisions should be recorded.

Format:

```
DATE:
DECISION:
WHY:
ALTERNATIVES:
IMPACT:
REVISIT WHEN:
```

Examples:

- Database selection
- Authentication provider
- HCP integration method
- Communication platform
- PWA vs native
- Hosting
- AI provider
- Offline architecture

---

20. RISK LOG

Track:

```
RISK:
IMPACT:
LIKELIHOOD:
MITIGATION:
OWNER:
STATUS:
```

High-risk issues should be discovered early.

---

21. DEPENDENCY LOG

Track external dependencies.

```
SERVICE:
PURPOSE:
COST:
FREE TIER:
CRITICALITY:
```

```
REPLACEMENT DIFFICULTY:
DATA STORED:
API:
STATUS:
```

This prevents hidden vendor lock-in.

---

22. CURRENT PROJECT DASHBOARD

Current Phase

Phase 0 — Discovery & Architecture

Current Objective

Create the final technical architecture and execution plan before implementation.

Current Status

NOT STARTED / ARCHITECTURE DEFINITION

Completed

- Overwatch vision established
- Technician-first direction established
- HCP remains initial system of record
- Job state model established
- Parts requirement established
- Pricing requirement established
- Part request requirement established
- Job outcome reasons established
- GPS/ETA requirement established
- Communication requirement established
- Offline requirement established
- Event-driven architecture direction established
- AI architecture direction established
- Security-by-design requirement established
- Two-branch Git workflow established
- Phase-gated development established
- Low-cost operating requirement established
- Code readability/commenting requirement established

In Progress

- Final architecture
- Technology selection
- HCP integration research
- Communication infrastructure research
- GPS strategy
- Offline strategy
- Authentication strategy
- Database strategy
- Hosting strategy
- First vertical slice definition

Blocked

None currently.

Known Risks

- HCP API limitations
- HCP synchronization behavior
- iOS PWA limitations
- Background GPS limitations
- Push notification limitations
- Offline synchronization complexity
- Communication infrastructure cost
- AI operating cost
- Data volume
- Employee/customer privacy
- Future scale

Next Major Step

Complete Phase 0 architecture and begin Phase 1 foundation.

---

23. PHASE COMPLETION RECORD

Every completed phase should be recorded.

```
PHASE:
OBJECTIVE:

FUNCTIONALITY:
```

```
[ ] Complete

SECURITY:
[ ] Complete

TESTING:
[ ] Complete

USER ACCEPTANCE:
[ ] Complete

DOCUMENTATION:
[ ] Complete

DEPLOYMENT:
[ ] Complete

PRODUCTION VERIFICATION:
[ ] Complete

KNOWN ISSUES:
None / Listed below

DECISIONS MADE:

DATE COMPLETED:

APPROVED TO MOVE FORWARD:
Yes / No
```

---

24. FAILURE PROTOCOL

If something does not work:

We do not immediately patch randomly.

Instead:

1. Reproduce the problem.
2. Determine where the failure occurs.
3. Identify the root cause.
4. Explain the cause.
5. Determine whether architecture is affected.
6. Fix the smallest correct layer.

7. Test the fix.
8. Regression-test affected functionality.
9. Document the issue if it could recur.
10. Continue only after the affected functionality is stable.

---

25. NO MAGIC CHANGES

The assistant should never silently:

- Change architecture
- Replace a library
- Change database structure
- Change authentication
- Change deployment
- Remove functionality
- Modify requirements

without explaining the reason.

If a change is necessary, document it.

---

26. DOCUMENTATION REQUIREMENTS

The project should eventually contain:

```
PROJECT_STATE.md
README.md
ARCHITECTURE.md
SECURITY.md
DATABASE.md
API.md
DEPLOYMENT.md
TESTING.md
```

Additional documentation can be created when justified.

---

27. PRODUCTION PHILOSOPHY

"Production" does not mean:

> It works on my computer.

Production means:

- Secure
- Tested
- Recoverable
- Documented
- Observable
- Usable
- Deployable
- Supported
- Verified

---

28. WHAT "SHIPPABLE" MEANS

Every phase must produce something that could reasonably be handed to its intended users or used as a stable foundation for the next phase.

A phase should not leave behind:

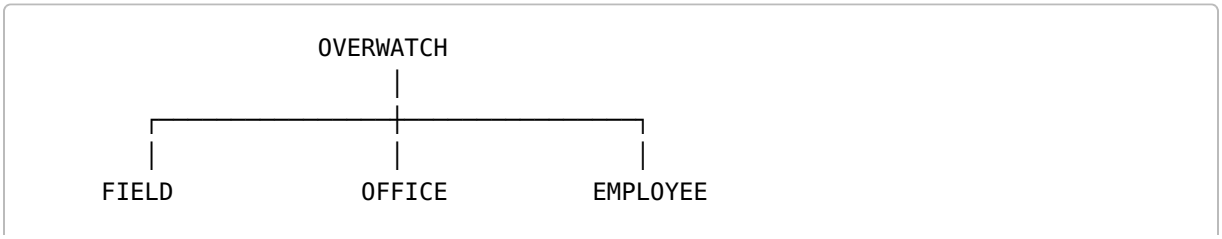
- Known critical bugs
- Unprotected APIs
- Broken permissions
- Unhandled obvious failures
- Undocumented architecture
- Untested critical workflows
- Temporary hacks that are being treated as permanent
- Data corruption risks

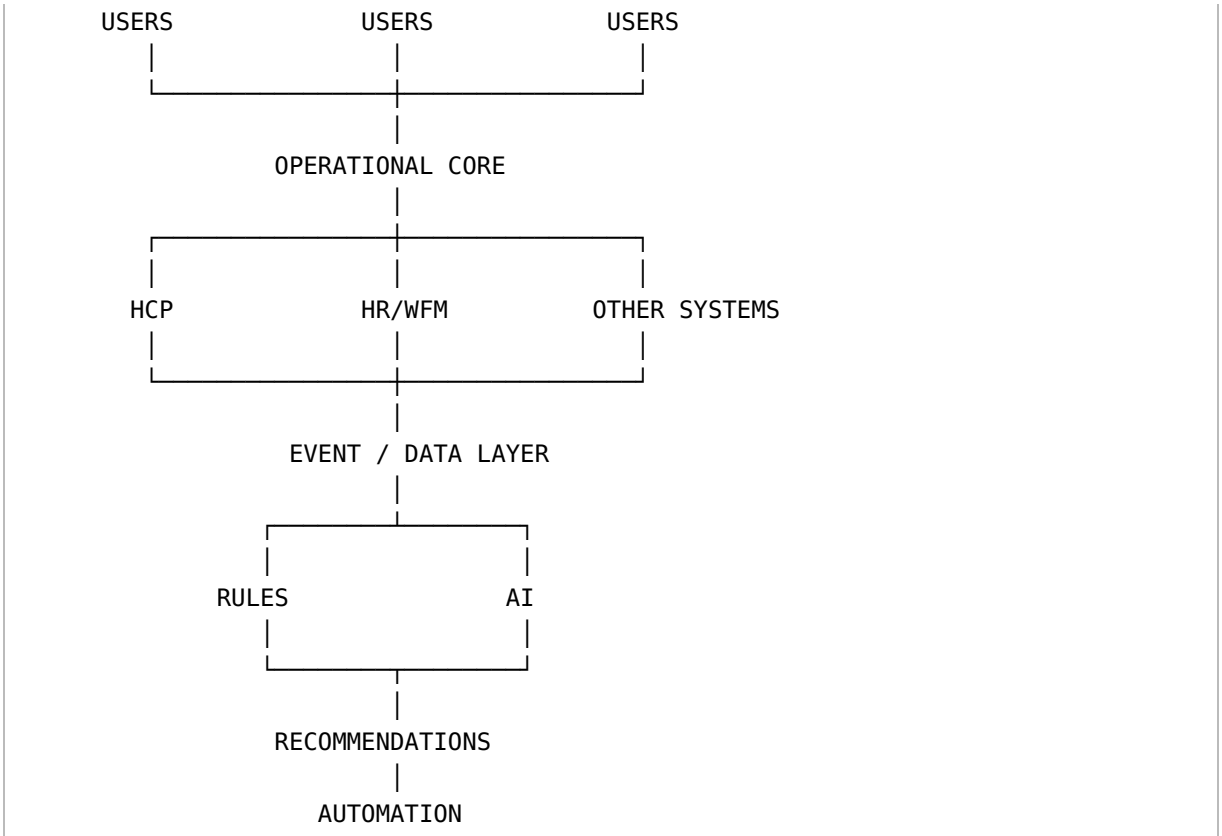
Temporary implementations are allowed during development only when clearly documented and intentionally scheduled for replacement.

---

29. LONG-TERM END STATE

The ultimate Overwatch vision is:





Over time, Overwatch should understand:

- What is happening
- What has happened
- What is likely to happen
- What needs attention
- What should happen next
- What action can safely be automated

---

30. ULTIMATE GOAL

Overwatch should eventually become the company's operational intelligence layer.

It should help answer:

> What is happening?
>
> Why is it happening?
>
> What is going to happen?

What should we do?

Can Overwatch prepare it?

Can Overwatch safely do it?

The ultimate goal is not simply to build another app.

The goal is to build a system that **understands the operation and helps the people running it make better decisions faster.**

---

31. CURRENT BUILD ORDER

The initial order is locked as:

```
0   Discovery & Architecture
1   Project Foundation
2   Identity & Permissions
3   HCP Integration Foundation
4   Technician My Day
5   Job State Machine
6   GPS & ETA
7   Job Documentation
8   Parts
9   Pricing
10  Part Requests / Office Assistance
11  Job Outcome Intelligence
12  Communication
13  Offline
14  Customer Experience
15  Event Bus
16  AI Intake
17  AI Watchdog
18  AI Assistant
19  Scheduling Intelligence
20  Operational Learning
21  Office Overwatch
22  Employee Portal
23  Automation
24  Auditability
25  Security Hardening
26  Complete Testing
27  UAT
28  Production Readiness
```

```
29  Deployment
30  Monitoring
```

Security, authentication, authorization, testing, documentation, and deployment requirements are **not actually deferred to their listed phases**.

They are built into every applicable phase.

Phases 25–30 represent formal hardening, verification, and production-readiness activities.

---

32. THE RULE FOR EVERY FUTURE FEATURE

Whenever a new feature is proposed, evaluate:

1. Why?

What problem does it solve?

2. Who?

Who uses it?

3. What data?

What information does it require?

4. Security?

Who is allowed to see/change it?

5. Integration?

Does another system need to know about it?

6. Offline?

Does it need to work without connectivity?

7. AI?

Does AI actually add value?

8. Cost?

Does it introduce an ongoing cost?

9. Testing?

How will we prove it works?

10. Phase?

Where does it belong?

11. Definition of Done?

What must be true before it is considered complete?

---

33. PROJECT COMPLETION DEFINITION

Overwatch is not "finished" when the original feature list is checked off.

The system is successful when:

- Technicians can perform their work efficiently.
- Office staff can see what is happening.
- Customers receive appropriate information.
- Information is entered once.
- HCP remains synchronized.
- Important operational events are visible.
- The company understands why work succeeds or fails.
- Scheduling improves from actual operational data.
- AI provides useful, grounded intelligence.
- Automation is controlled and auditable.
- Employees have a unified operational experience.
- The system is secure.
- The system is reliable.
- The system can evolve without becoming unmaintainable.

---

34. GOLDEN RULES

These rules override convenience.

Rule 1

Do not skip a phase gate.

Rule 2

Security is built in from the beginning.

Rule 3

A feature is not done until it is tested and accepted.

Rule 4

Do not build months of functionality before proving the architecture works.

Rule 5

Prefer simple, understandable code over clever code.

Rule 6

Comment important code so the project owner can learn from it later.

Rule 7

Do not introduce unnecessary paid services.

Rule 8

Do not let AI become the source of truth for deterministic information.

Rule 9

Never silently change architecture or requirements.

Rule 10

Every important operational action must be explainable and auditable.

Rule 11

A job that does not get completed must have a known reason and appropriate next action.

Rule 12

Technicians should enter information once whenever reasonably possible.

Rule 13

HCP remains the initial system of record unless a deliberate architectural decision changes that.

Rule 14

The roadmap can change, but changes must be intentional and documented.

Rule 15

The goal is not to write the most code. The goal is to build the correct system.

---

35. STARTING POINT

Current Status

PHASE 0 — DISCOVERY & ARCHITECTURE

We are NOT coding the full application yet.

The immediate objective is to finish the architecture decisions necessary to safely begin Phase 1.

First implementation target

The first real vertical slice will be:

```
Secure Technician Login
        ↓
Today's Jobs
        ↓
Open Job
        ↓
View Job Information
        ↓
Change Job State
        ↓
Persist Change
        ↓
Create Event
        ↓
Audit Change
```

```
        ↓
Synchronize With HCP
```

Once this slice works end-to-end, securely, locally, through `dev`, through testing/UAT, and through deployment verification, we have proven the foundation.

Then we build the next slice.

---

36. PROJECT STATE UPDATE RULE

At the end of every meaningful development session, PROJECT_STATE.md should be updated when the state has materially changed.

At minimum, update:

- Current phase
- Current objective
- Completed work
- In-progress work
- Blockers
- Known issues
- Decisions
- Security status
- Testing status
- Next step

This prevents the project from losing its place.

FINAL PROJECT PRINCIPLE

Build it correctly.

Secure it as we build it.

Test it before moving on.

Make it understandable.

Document the decisions.

Ship each phase.

Then build the next piece.

Never sacrifice the foundation just to move faster.